

Lezione 9 20/11/23 GOF2

giovedì 23 novembre 2023 13:41

Continuiamo il discorso sui pattern : andiamo ora a vedere gli altri :

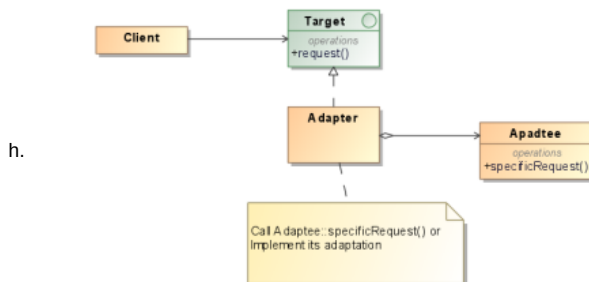
		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method (107)	Adapter (139)	Interpreter (243) Template Method (325)
	Object	Abstract Factory (87) Builder (97) Prototype (117) Singleton (127)	Adapter (139) Bridge (151) Composite (163) Decorator (175) Facade (185) Proxy (207)	Chain of Responsibility (223) Command (233) Iterator (257) Mediator (273) Memento (283) Flyweight (195) Observer (293) State (305) Strategy (315) Visitor (331)

La scorsa lezione , abbiamo visto quelli creazionali , ora andiamo a vedere quelli **strutturali** : adapter, decorator e facade . Questi pattern sono caratterizzati dalla caratterizzazione/descrizione della modalità di interazione tra le classi , per arrivare alla formazione e interazione di strutture complesse . Quindi si basano su ereditarietà delle classi , facendo sì che vi sia composizione tra loro per arrivare a formare interfacce o implementazioni complesse , come per esempio si combinano due o più classi con l'ereditarietà multipla : si cerca di creare una classe che raccoglie tutte le caratteristiche delle super-classi. Mentre per quello che riguarda **gli oggetti**, questi pattern modellano la composizione tra oggetti per realizzare nuove funzionalità , aumentando così la flessibilità , in quanto hanno una struttura dinamica. In dettaglio :

1. Adapter

- Lo scopo di questo pattern è di gestire interfacce incompatibili, fornendo un'interfaccia stabile a classi funzionalmente simili o con interfacce diverse.
- Chiamato anche Wrapper
- Questo pattern viene usato in quanto, in generale , in un qualunque sistema sw, si cerca di progettare/strutturare le classi in modo che vi sia riuso. Ma di solito è difficile che si ha la riusabilità tra le classi : l'interfaccia esportata non rispecchia i requisiti .
- Vediamolo in dettaglio : si ha un sistema nel quale si vuole usare una classe esistente, ma la sua interfaccia non è compatibile, quindi si deve per forza realizzare una classe che fa da tramite . Così si arriva alla cooperazione tra le varie classi/parti del sistema incompatibili tra loro .
- Notiamo che i clients invocano operazioni su istanza di adapter, la quale classe (adapter) gestisce opportunamente invocazioni verso istanze di Adaptee.
- Se in presenza di ereditarietà (Adaptee), la classe Adapter potrebbe non svolgere correttamente il "fare da ponte" : dipende tutto da quanto sono simili le interfacce alle quali serve "il ponte".
- Questo pattern non è sempre trasparente al client : non è sempre possibile usarlo .**

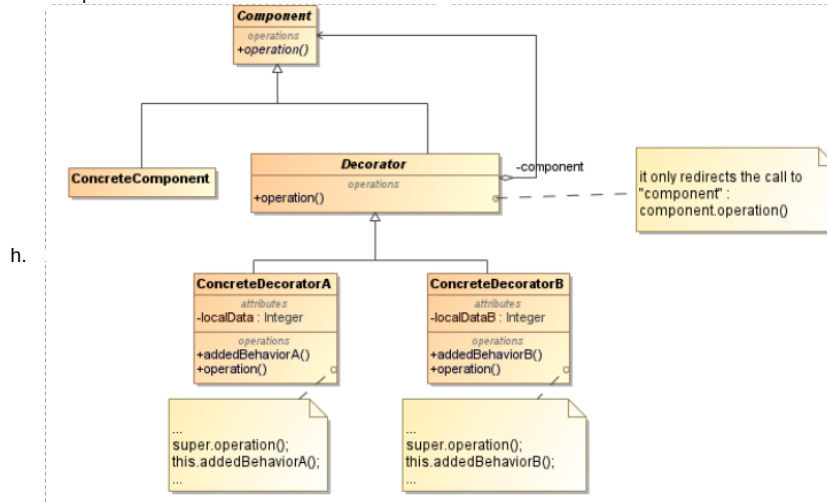
• Struttura



2. Decorator :

- Forniscono nuove funzionalità al sistema : consentono di aggiungere nuove responsabilità ad una classe ; Quindi aggiungendo queste responsabilità, si ha una flessibile alternativa alla definizione di sottoclassi : estensione della funzionalità delle classi .
- Anche questo chiamato Wrapper
- Si usa ereditarietà per aggiungere nuove funzionalità .**
- Quindi usando l'ereditarietà si arriva a questo scenario : vi è una sola classe che ha la responsabilità di aggiungere comportamenti : questa classe racchiude il componente da decorare .
- L'oggetto contenitore viene chiamato **decorator** ; il quale ha interfaccia conforme all'oggetto decorato , facendo così sì che abbia trasparenza verso il client . Le richieste di questo "oggetto" vengono mandate al componente decorato , effettuando così azioni aggiuntive prima della richiesta. **Possibile annidare più decorator per aggiungere responsabilità.**
- Quindi si aggiungono responsabilità agli oggetti in modo dinamico e trasparente ; diminuisco quindi le responsabilità delle singole classi, e **permette di bypassare la definizioni di sottoclassi.**
- Riassumendo** : il decorator trasferisce richieste al suo component. Eventualmente può svolgere altre azioni anche prima e dopo il trasferimento della richiesta.

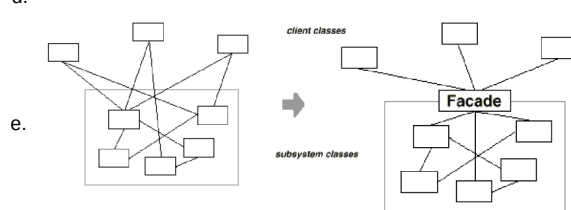
Aumenta quindi la flessibilità rispetto all'ereditarietà multipla statica : aggiunta/rimozione decoratori facile , evitando così di creare gerarchie troppo complesse .



3. Facade :

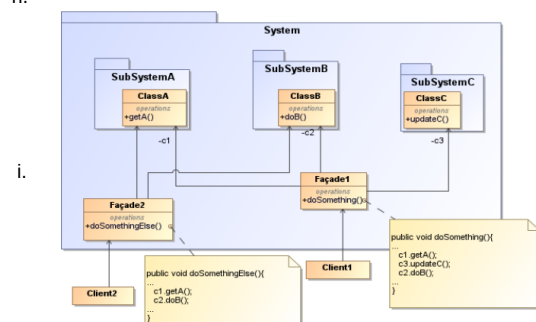
- Consente di fornire interfaccia stabile ad un insieme di interfacce di un sottosistema;
- Non ha sinonimi
- Vista l'interfaccia stabile ed unificata per un insieme disparato di implementazioni , permette di minimizzare le comunicazioni e le dipendenze tra sottosistemi , mascherando così all'utente l'implementazione del sottosistema (implementazioni variabili ma funzionalità inalterate).

d.



- Quindi : serve per fornire interfaccia **unica** a sistema che evolve nel tempo, aumenta il grado di riuso del sottosistema , **stratifica il sistema individuando tutti i punti di accesso** : aumenta così il numero di dipendenze tra il sottosistema ed il client , in quanto il client deve accedere a più classi diverse del sottosistema.
- Riassumendo** : i client comunicano con il sistema attraverso l'interfaccia di Facade , abolendo così la possibilità di accesso agli oggetti del sistema da parte dei client. Si ha quindi una riduzione del numero di oggetti che un client deve gestire, aumenta il riuso , **diminuisce il grado di accoppiamento (ripercussione minima dei cambiamenti del sistema sui client)**, diminuzione delle dipendenze circolari .

h.



Vediamo ora l'ultima categoria dei pattern : **comportamentali** : definiscono modi di comunicazione tra le classi e gli oggetti coinvolti nel pattern.

1. Observer :

- Questo pattern serve a definire una corrispondenza uno a molti tra oggetti : se un oggetto cambia il suo stato , tutti gli oggetti dipendenti da quest'ultimo vengono notificati ed aggiornati automaticamente .
 - Sinonimi : Dependents, Publish-Subscribe
 - Vista l'interoperabilità di classi differenti su unica sorgente dati / istanza classe comune , le istanze di queste classi devono essere notificate se istanza classe comune cambia .
 - In generale o meglio se applicazione complessa si dovrebbe garantire la separazione tra l'interfaccia grafica e la sottostante struttura dell'applicazione ; quindi vi dovrebbe essere una sorta di "riusabilità indipendente" ma comunque un minimo devono cooperare.
 - Visti gli aspetti dipendenti di un'unica entità, grazie a questo pattern , li possiamo incapsulare in classi separate ed indipendenti .
 - Permette di gestire la modifica della classe base , notificando gli osservatori (anche se in numero sconosciuto), **mantenendo un alto livello di disaccoppiamento**.
 - Riassumendo** : con questo pattern è possibile variare soggetti ed osservatori in modo indipendente : possibile riuso soggetti senza usare gli osservatori : **accoppiamento tra subject ed observer minimale**, supporto comunicazioni broadcast , ma attenzione agli aggiornamenti inattesi : opportuna gestione.
- h.

